

EE 224

IITB - CPU

Project Task - 2
Group - 2

Prof : Virendra Singh

TEAM MEMBERS:

Prathmesh Santosh Baral (23B1260)

Maitranya Mahesh Patil (23B1239)

Piyush Manish Gube (23B1222)

Vishnu Bijarniya (23B1251)

Introduction

We have made an IITB-CPU capable of performing 15 operations (R, I and J type). For storing data we have used 8 registers (R0 to R7) of 16 bits each. The language we have used is VHDL.

Components

To achieve our CPU various components are required. Next given are the important ones:

- **Register File (RF) :**

It is used to read and write in registers R0 to R7.

For reading, we use two 8x1 16 bit mux with inputs being the 8 registers and outputs D1 & D2.

Which register to read is selected using select lines A1 & A2 of those muxes.

For writing we have used a 1x8 16 bit demux with input D3 data to be stored and output 8 registers which is selected using A3. To control when to write we use enable and using another 1x8 1 bit demux, we give enable to the selected register.

- **ALU:** Our ALU has two 16-bit inputs and one 16-bit output and three select lines P2 P1 P0 which choose which operation to perform. Two more outputs are

C-Flag which is one if there is carry and Z-flag which is one if ALU_C is zero.

It is used to perform ADD, SUB, MUL, IMP, OR, AND that is all R -type. The individual operations are performed inside using an adder-subtractor, 4-bit multiplier and logic gates respectively.

- **Memory:** It stores instructions of 16 bits at the memory addresses. First we convert the PC value of 16 bits into unsigned integer and then at that point, and next one we get total 16 bits with next one being in MSB. This 16 bit instruction is given out from mem_out. To read and write we use mem_write and mem_read controls. We have also stored some instructions at the addresses to check the working of our CPU.
- **Register:** Program counter(PC), Instruction Register(IR), Temporary Registers (T1,T2) and Registers (R0 to R7) are all Registers which are actually D Flip flops storing some 16 bit value.

I] PC : It points at the instruction where we are in memory hence it is also called Instruction pointer. It is incremented by 2 to go in further instructions.

II] IR: It stores the instruction received from memory. The instruction is of 16 bits where the first 4 bits denote the operation to be performed.

III] T1, T2 : They are used for storing some value temporarily which is used for some specific function.

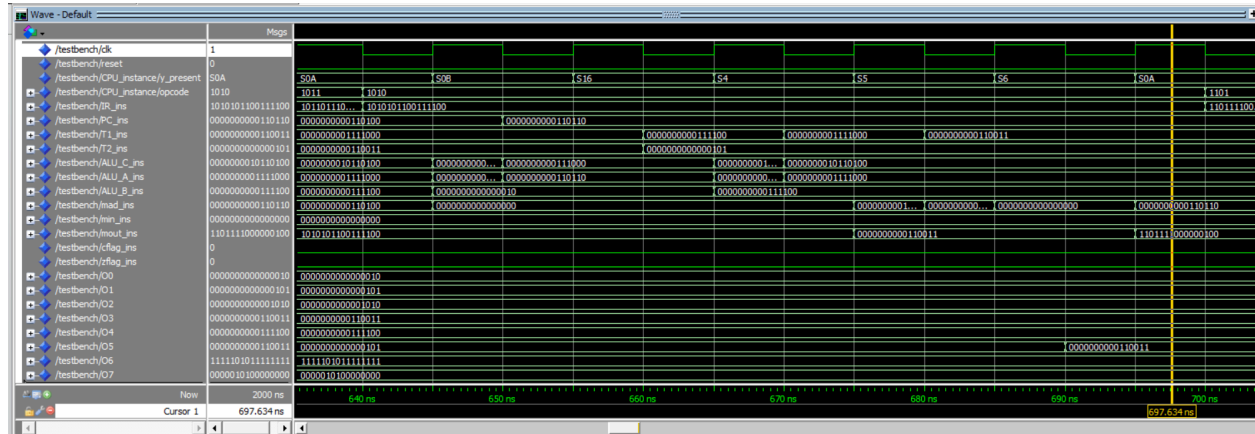
- **1s shift :** It acts as a left shifter i.e. it multiplies by 2 which shifts the binary number one bit left.
- **SE 16 :** It is used to extend the number of bits to 16 by adding zeros in starting.

Points to consider:

- We wrote S0 in two parts S0a and S0b as we did in jumps, we didn't want to increment PC by 2 as the jumps were from the instruction address itself.
- We didn't use mux as shown in pen paper as while coding each input served as some output which gave multiple constant driver errors
- We introduced S16 as in load and store the RF_A1 was to be stored in T2 and RF_A2 in T1 which was just opposite to S1.
- In FSM, we have used a moore type machine because the outputs depend on our states only like s1,s9,etc. According to inputs and current state, we just change the states.
- So if reset is excluded we have states S0a,S0b and S1 to S16 that is a total of 18 states.

Now let's look at some specific instructions execution

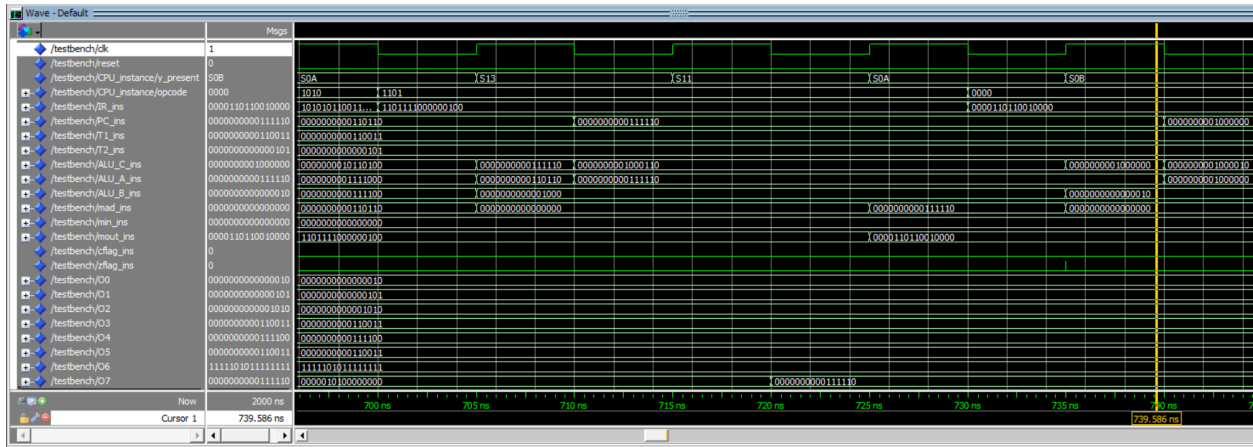
LOAD:



So, first PC (0..00110100) which is 52 comes at mad(memory address) and mem gives the instruction corresponding to it in m_out which is then stored in IR. This is state s0a. Now in s0b, PC goes in ALU_A and +2 is given at ALU_B, ALU_C is stored in PC and it goes to the next instruction. Now as IR is 1010 101 100 111100 it tells to store at R5 and the address is computed using R4 and Imm 111100.

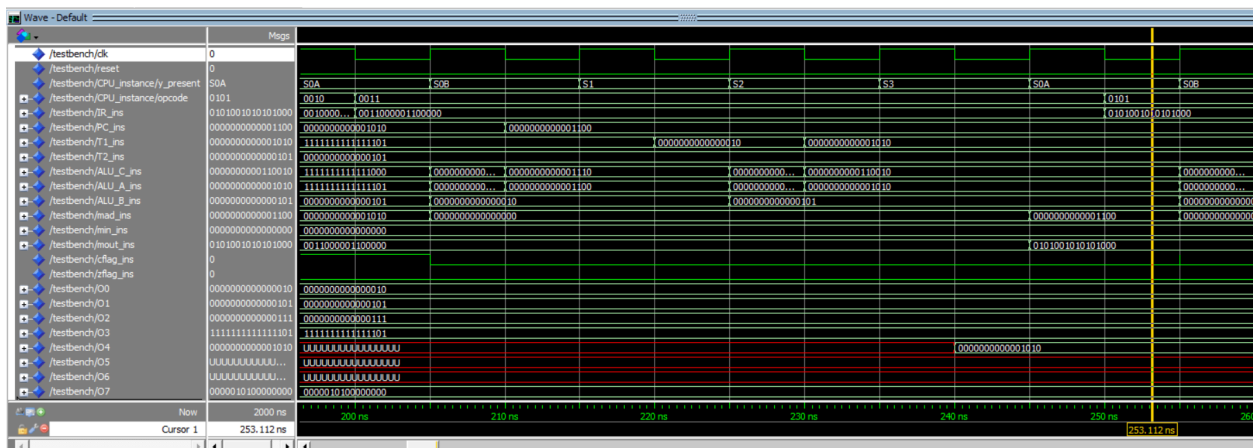
In S16 we store in T1 value from R4 and in T2 from R5. Then in S4, we find the address by giving T1 in ALU_A, IMM in ALU_B and ALU_C back in T1. Now in S5 T1 goes at that address which here is 120 and stores mem out which is 51 (which i had stored in previous step). Now in S6 this value is written in R5. Hence load operation is completed and now CPU performs the next operation.

JAL



PC storing 54 goes at mem_add and the instruction there is stored at IR. IR stores 1101 111 00000100 So S0a gets completed and s9 starts where PC goes in ALU_A and IMM after SE and 1S in ALU_B and ALU_C is stored in PC. After that in S11 this value of PC is stored in R7(111). After that we go back to state S0a.

MUL



Firstly S0a and Sob states occur as in load. The IR value here is 0011 000 001 100 000. So in S1 we read in T1 and T2 from R0 and R1. In S2 we feed them in ALU_A and ALU_B respectively and IR 14-12 which is 0011 is used to select mul, ALU then multiplies last 4 bits of inputs and gives output ALU_C which is stored in T1. In S3 T1 goes to R4(100) and stores this value in it and then we go back in S0a for next instruction.

STATE DESCRIPTION

S0a	<table border="1"> <tr> <td> $PC \rightarrow Mem_Add$ $Mem_D \rightarrow IR$ </td><td> Mem_Read IR_en </td></tr> </table>	$PC \rightarrow Mem_Add$ $Mem_D \rightarrow IR$	Mem_Read IR_en
$PC \rightarrow Mem_Add$ $Mem_D \rightarrow IR$	Mem_Read IR_en		

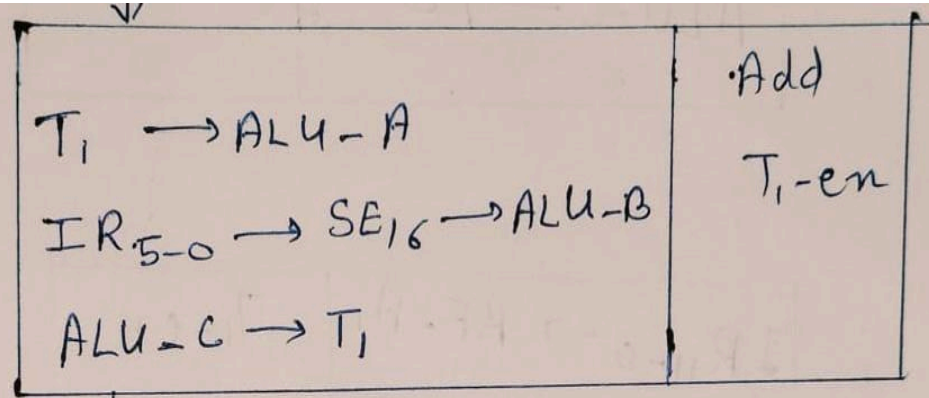
S0b	<table border="1"> <tr> <td> $PC \rightarrow ALU-A$ $+2 \rightarrow ALU-B$ $ALU-C \rightarrow PC$ </td><td> ADD PC_en </td></tr> </table>	$PC \rightarrow ALU-A$ $+2 \rightarrow ALU-B$ $ALU-C \rightarrow PC$	ADD PC_en
$PC \rightarrow ALU-A$ $+2 \rightarrow ALU-B$ $ALU-C \rightarrow PC$	ADD PC_en		

S1	<table border="1"> <tr> <td>$IR_{11-9} \rightarrow RF-A_1$</td><td>$T_1_en$</td></tr> <tr> <td>$IR_{8-6} \rightarrow RF-A_2$</td><td>$T_2_en$</td></tr> <tr> <td>$RF-D_1 \rightarrow T_1$</td><td></td></tr> <tr> <td>$RF-D_2 \rightarrow T_2$</td><td></td></tr> </table>	$IR_{11-9} \rightarrow RF-A_1$	T_1_en	$IR_{8-6} \rightarrow RF-A_2$	T_2_en	$RF-D_1 \rightarrow T_1$		$RF-D_2 \rightarrow T_2$	
$IR_{11-9} \rightarrow RF-A_1$	T_1_en								
$IR_{8-6} \rightarrow RF-A_2$	T_2_en								
$RF-D_1 \rightarrow T_1$									
$RF-D_2 \rightarrow T_2$									

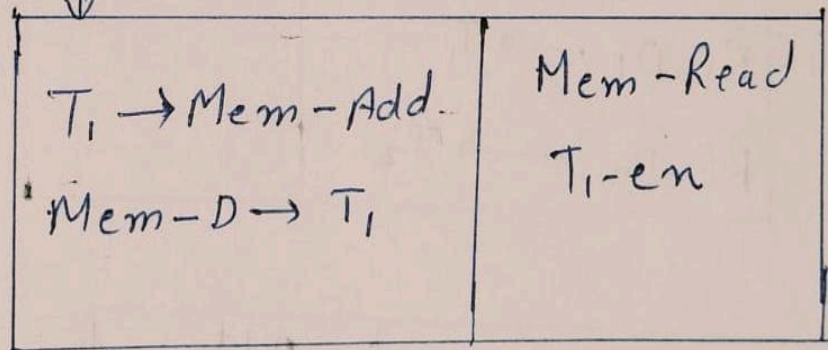
S2	<table border="1"> <tr> <td> $T_1 \rightarrow ALU-A$ $T_2 \rightarrow ALU-B$ $ALU-C \rightarrow T_1$ </td><td> ADD SUB MUL IMP AND OR T_1_en </td></tr> </table>	$T_1 \rightarrow ALU-A$ $T_2 \rightarrow ALU-B$ $ALU-C \rightarrow T_1$	ADD SUB MUL IMP AND OR T_1_en
$T_1 \rightarrow ALU-A$ $T_2 \rightarrow ALU-B$ $ALU-C \rightarrow T_1$	ADD SUB MUL IMP AND OR T_1_en		

S3	<table border="1"> <tr> <td> $T_1 \rightarrow RF-D_3$ $IR_{5-3} \rightarrow RF-A_3$ </td><td> RF_en </td></tr> </table>	$T_1 \rightarrow RF-D_3$ $IR_{5-3} \rightarrow RF-A_3$	RF_en
$T_1 \rightarrow RF-D_3$ $IR_{5-3} \rightarrow RF-A_3$	RF_en		

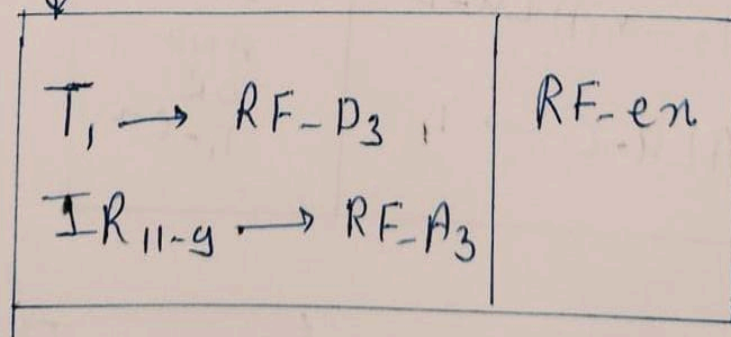
S₄



S₅



S₆



S₇

T ₁ → Mem-Add	Mem-write
T ₂ → Mem-Data	

S₈

T ₁ → ALU-A	SUB
T ₂ → ALU-B	

S₉

PC → ALU-A	Add PC-en
IR ₅₋₆ → SE ₁₆ → IS → ALU-B	
ALU-C → PC	

S₁₀

T ₁ → RF-D ₃	RF-en
IR ₈₋₆ → RF-A ₃	

S₁₁

PC → RF-D ₃	RF-en
IR ₁₁₋₉ → RF-A ₃	

S₁₂

IR ₈₋₆ → RF-A ₂	PC-en
RF-D ₂ → PC	

S₁₃

$PC \rightarrow ALU-A$ $IR_{8-0} \rightarrow SG_{16} \rightarrow 1S \rightarrow ALU-B$ $ALU-C \rightarrow PC$	ADD PC-en
---	--------------

S₁₄

$IR_{7-0} \rightarrow 1S (8 \text{ times}) \rightarrow RF-D_3$ $IR_{11-9} \rightarrow RF-A_3$	RF-en
--	-------

S₁₅

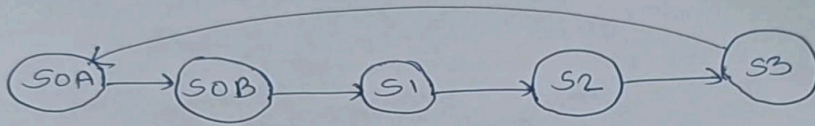
$IR_{7-0} \rightarrow SG_{16} \rightarrow RF-D_3$ $IR_{11-9} \rightarrow RF-A_3$	RF-en
---	-------

S₁₆

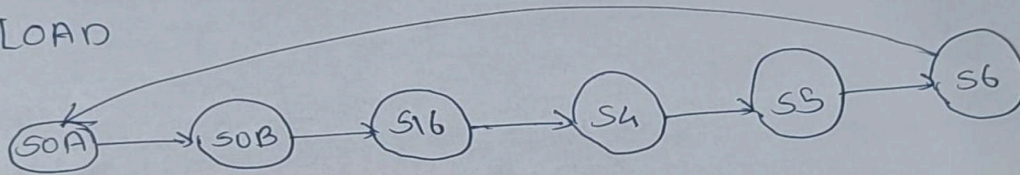
$IR_{11-9} \rightarrow RF-A_1$ $IR_{8-6} \rightarrow RF-A_2$ $RF-D_1 \rightarrow T_2$ $RF-D_2 \rightarrow T_1$	T_1-en T_2-en
---	----------------------

STATE FLOW

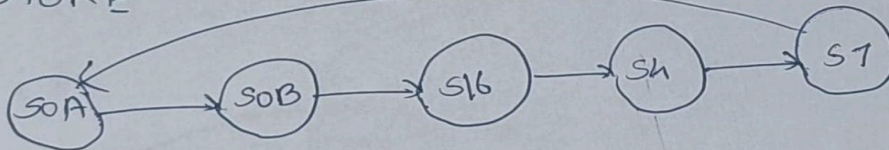
ADD, SUB, MUL, AND, ORA, IMP



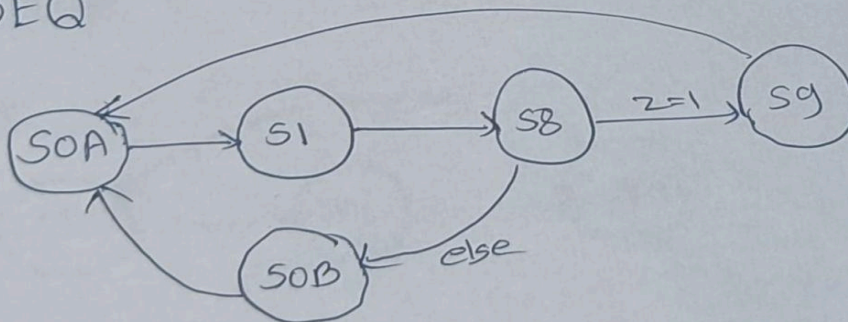
LOAD



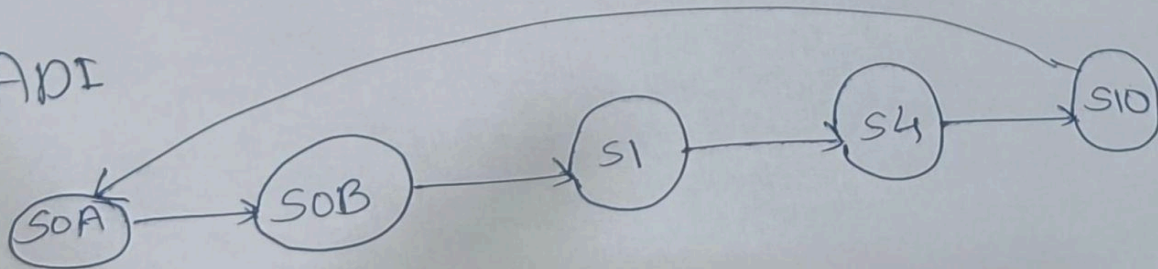
STORE



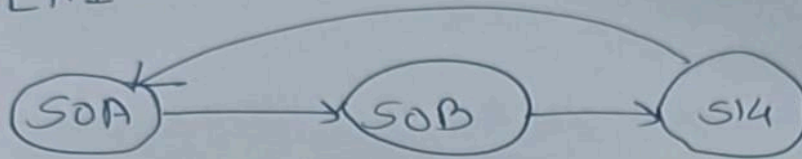
BEQ



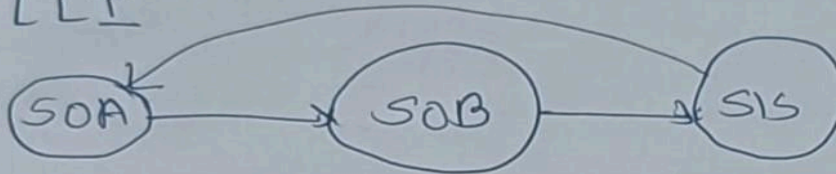
ADI



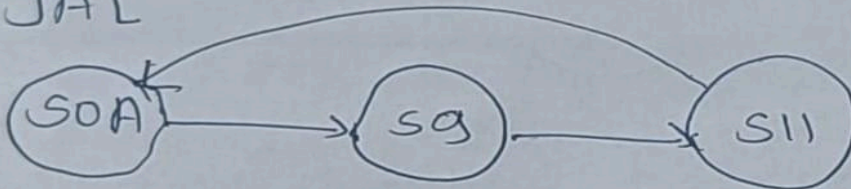
LHI



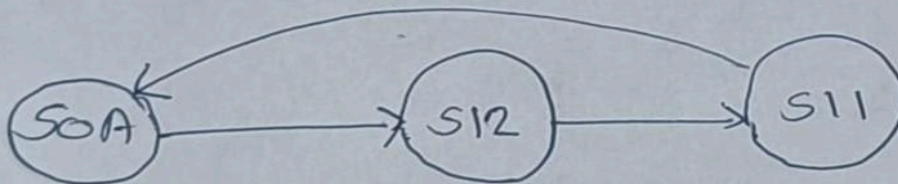
LLI



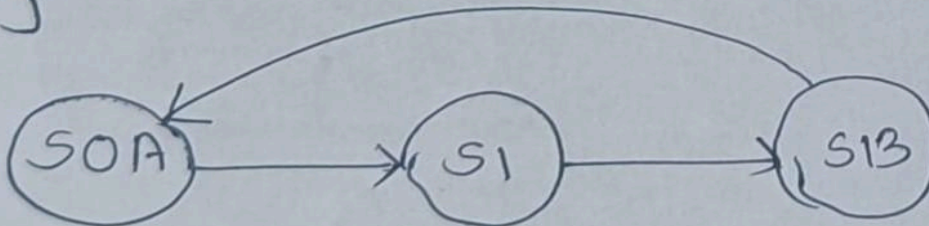
JAL



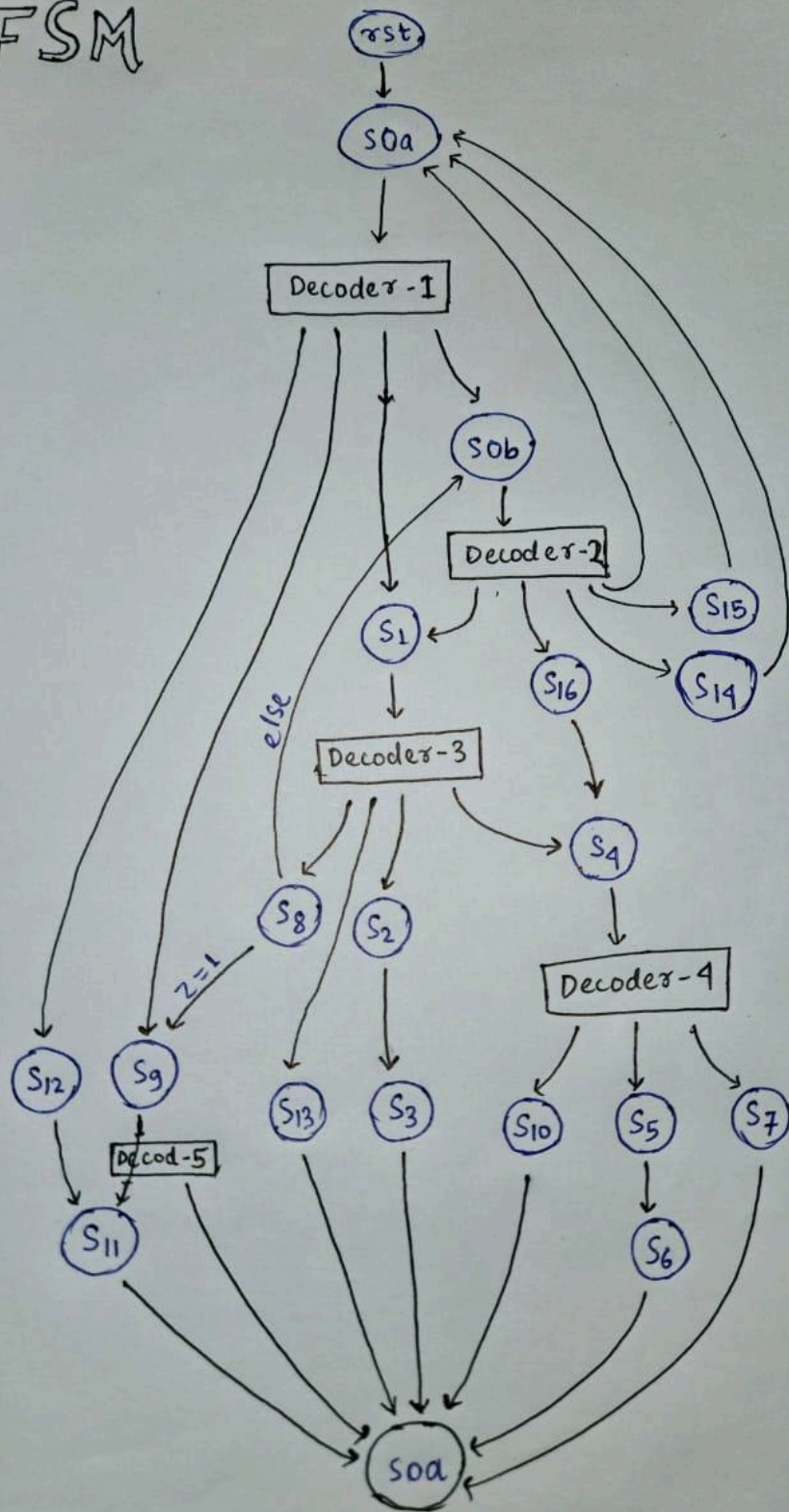
JLR



J



FSM



Work Distribution :

- Prathmesh S. Baral : 23B1260
Did coding for CPU which included FSM and integrating all components. Also did coding for testbench.
- Maitranya M. Patil : 23B1239
Made the state flow diagram, state description, and pen paper FSM. Did coding for ALU, register & register file.
- Piyush M. Gube : 23B1222
Did coding for all types of mux, sign extension, shifts, dff, and, or, imp, full adder, multiplier and made the reports.
- Vishnu Bijarniya : 23B1251
Ideated and made the circuit diagram and truth table. Coded memory & checked all outputs and correct execution of CPU.